AstraNav: Design and Experimental Analysis of Shortest-Path Algorithms for Campus Navigation

Swagatam Behera (AP25110070661) , Katragadda Nehasri (AP25110070846)
Dept. of Computer Science & Engineering,
SRM University–AP, Andhra Pradesh, India
Guide: Prof. Srinivas Arukonda, Assistant Professor, Dept. of CSE

***Abstract*—The purpose of AstraNav is to explore campus route-finding using a campus map representation as a schematic graph and comparing Dijkstra's algorithm with Bidirectional Dijkstra's. The application consists of a JSON graph, a C++17 shortest path solver, Python FastAPI back-end server, and front-end running on browser, visualizing the calculated route on the map of the campus. The experiments were performed on all 1,260 ordered pairs of sources and destinations in a graph with 36 vertices, with ten executions per each pair and 10,000 repeats in-process for each time calculation. Both methods produced valid paths in each experiment and provided the same reachability and the same cost of the path. Dijkstra's method settled on average 19.000 vertices and had an average execution time of 0.001204 ms, while Bidirectional Dijkstra settled on average 13.594 vertices and had an average execution time of 0.001566 ms. Therefore, bidirectional searching decreased the number of settled vertices but increased execution time in this case. Graph edge weights are representative kilometers, not measured pedestrian paths.**

***Index Terms*—Shortest-path algorithms, Dijkstra's algorithm, Bidirectional Dijkstra, campus navigation, graph algorithms, algorithm analysis.**

I. Introduction

Finding a path between two points is a traditional graph-search task. In the campus context, buildings and pedestrian intersections could be represented as vertices, while traversable links between them as weighted edges. Then, the route planning task amounts to finding a path connecting a chosen source and destination with the minimal total cost.

AstraNav is a didactic project that is meant to show an example of applying and analyzing shortest-path algorithms in the context of a campus. It is not meant to be used as a reliable navigation system. However, it demonstrates the whole pipeline of such system including representation of a graph in JSON format, shortest-path calculation by a C++17 engine, API for sending requests using FastAPI framework and visualization of the calculated path on a webpage.

In the course of this project, two algorithms are investigated: classic Dijkstra and bidirectional Dijkstra. The former is one of the basic label-setting algorithms for non-negative edge costs [1]. The latter algorithm runs a search both from the source and from the destination; its advantage depends on the nature of the graph and the termination condition. Shortest-path search is still an active area of research [3].

The contributions of this project are:

- A 36-vertex, 39-edge schematic campus pedestrian graph represented in JSON, with all encoded edges marked bidirectional.
- C++17 implementations of Dijkstra and Bidirectional Dijkstra, integrated with a FastAPI backend and browser-based route visualization.
- A repeatable all-pairs benchmark covering 1,260 ordered source–destination queries, with path validation, shortest-cost comparison, settled-vertex counts, and execution-time measurements.
- An empirical comparison showing that Bidirectional Dijkstra settled fewer vertices on average, while Dijkstra had lower measured execution time on this graph.

The coordinates used for the graph come from the schematic reference map. Before implementation of the graph in the real world, everything should first be checked separately.

II. Related Work

The label-setting approach for finding shortest paths in networks with non-negative edge costs was developed by Dijkstra [1]. Efficient selection of the next unsettled node requires the use of priority queues, which selection can influence the time complexity of the algorithm; the work of Fredman and Tarjan investigated Fibonacci heaps and their use for network optimization [2].

Bidirectional search is the point-to-point approach in which searches occur from both sides simultaneously. The performance of this approach is influenced by the topology of the graph, the choice of expansion rule, and a proper termination condition. Goldberg and Harrelson studied point-to-point shortest-path computation and the relationship between A* search and graph-theoretic methods [3]. Bast et al. provide a broader survey of route-planning methods and shortest-path techniques [4].

AstraNav makes no claims about a novel shortest-path algorithm. Its main contribution is combining and comparing two known algorithms on the graph with campus-like topology along with the graphical user interface.

III. Problem Formulation

Let the encoded campus network be represented by a weighted undirected graph

$$G = (V, E, w), \tag{1}$$

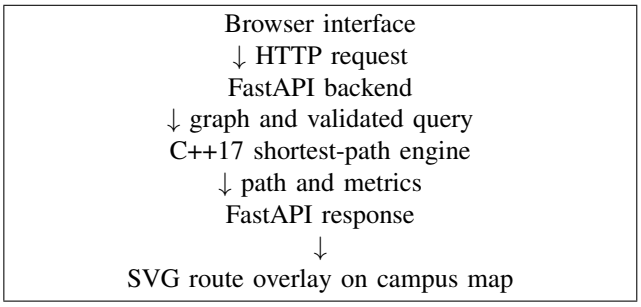


Fig. 1. AstraNav request and computation pipeline.

where V is the set of vertices, E is the set of edges, and $w : E \to \mathbb{R}_{\geq 0}$ assigns a non-negative weight to each edge.

For a source vertex $s \in V$ and destination vertex $t \in V$, the objective is to find a path

$$P(s,t) = (v_0 = s, v_1, \ldots, v_k = t) \tag{2}$$

that minimizes

$$W(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}). \tag{3}$$

A valid path must satisfy $(v_i, v_{i+1}) \in E$ for every consecutive pair. If no such path exists, the query is reported as unreachable.

The frozen graph used for the reported experiment contains 36 vertices and 39 edge records. Every edge record is marked bidirectional, so each encoded connection is traversable in both directions. The graph is derived from a schematic campus reference map; node coordinates and walkway connectivity are schematic and have not been surveyed. Edge weights are illustrative kilometres and must not be interpreted as measured walking distances. The SHA-256 hash of the graph file used to identify this graph version is `46a2f7ce089f837c` (first 16 hexadecimal characters).

IV. System Architecture

AstraNav consists of four main components:

1) **Frontend:** HTML, CSS, and JavaScript provide source and destination selection, algorithm selection, and route display.
2) **Backend:** FastAPI exposes endpoints for graph retrieval, health checking, and route requests.
3) **Algorithm engine:** A compiled C++17 executable runs the selected shortest-path algorithm.
4) **Graph data:** A JSON file stores node identifiers, names, coordinates, and weighted edges.

The order of request is: frontend provides source, destination and algorithm IDs to the FastAPI backend; FastAPI backend verifies the request and runs C++ engine; C++ engine responds with a path and some metrics; FastAPI backend verifies the resulting path; frontend renders the resulting sequence of nodes on top of the campus map.

V. Algorithm Design

A. Dijkstra's Algorithm

Dijkstra's algorithm maintains a tentative distance $d[v]$ for every vertex. Initially, $d[s] = 0$ and all other distances are infinity. At each iteration, the unsettled vertex with the smallest tentative distance is selected. For every adjacent vertex v, the relaxation operation is

$$\text{if } d[u] + w(u,v) < d[v], \quad d[v] \leftarrow d[u] + w(u,v). \tag{4}$$

Predecessor array is maintained to store the path that is used to construct the output from t to s.

The traditional running time complexity using adjacency list data structure and binary min priority queue is $O((|V| + |E|) \log |V|)$, usually stated as $O(|E| \log |V|)$ for connected sparse graphs. The storage complexity for the graph is $O(|V| + |E|)$, while the distance array, predecessor array, and priority queue need extra memory.

B. Bidirectional Dijkstra

Bidirectional Dijkstra performs searches in both directions; one forwards from s and one backwards from t. The implementation maintains separate forward and backward distance arrays, a forward predecessor array, a backward successor array, settled sets, and two min-priority queues. The backward search operates on the reverse adjacency representation. At each iteration, the frontier with the smaller minimum tentative distance is expanded. Whenever the searches establish a complete connecting path, the best known path cost μ and its connection information are updated. Search terminates when the sum of the minimum keys in the two queues is greater than or equal to μ, or when no further connecting path can be found. The final path is reconstructed from the forward predecessor and backward successor information. The implementation's explored-vertex count is the sum of vertices settled by the two searches.

The worst-case asymptotic complexity is still $O((|V| + |E|) \log |V|)$ using binary heaps. Bidirectional search can help to shrink the explored space in certain graphs, but not necessarily to find fewer explored vertices or less elapsed time per query.

VI. Implementation

The algorithm engine is written in C++17. The graph is stored in JSON and loaded by the Python backend. FastAPI provides the application endpoints:

- `GET /api/health`: reports backend and engine status.
- `GET /api/graph`: returns graph data for the interface.
- `POST /api/route`: requests a route for a source, destination, and algorithm.
- `GET /docs`: provides interactive API documentation.

Front-end contains a map of the campus and depicts the resulting route using an SVG polyline. The accurate mapping of the path implies that the SVG coordinate system should match the coordinate system of the graph. It may happen that a path is mathematically correct based on the stored graph,

Algorithm 1 High-level bidirectional Dijkstra outline

Require: Weighted undirected graph G, source s, target t

Initialize forward and backward distances, parents, and settled sets
Initialize min-priority queues Q_f and Q_b
$\mu \leftarrow \infty$; store the best connecting path
while Q_f and Q_b are nonempty **do**
    if $\min(Q_f) \leq \min(Q_b)$ **then**
        Extract and settle the minimum-key forward vertex; relax its edges
    else
        Extract and settle the minimum-key backward vertex; relax reverse edges
    end if
    Update μ and connection data whenever a complete path is found
    if $\min(Q_f) + \min(Q_b) \geq \mu$ **then**
        break
    end if
end while
Reconstruct the path from forward parents and backward successors
return path and total cost, or unreachable

but geographically incorrect due to the topology of the graph and its nodes being inconsistent with the real-life paths on the ground.

VII. Experimental Methodology

A. Dataset and Graph Validation

The experiment uses the frozen 36-vertex, 39-edge graph identified by the hash reported in Table I. All 39 JSON edge records are bidirectional. The graph is based on a schematic campus reference map, and its edge weights are illustrative kilometres rather than surveyed distances. The following structural and route checks were applied:

- every edge endpoint refers to an existing vertex;
- all weights are finite and non-negative;
- no unintended duplicate edges exist;
- every displayed route consists only of graph edges;
- the visual coordinates align with the corresponding walkway junctions.

The graph can only be called a dataset of actual walking distances when the weights have been measured or calibrated.

B. Queries and Metrics

The query set contains every ordered pair of distinct vertices. Therefore, $Q = |V|(|V|-1) = 36 \times 35 = 1,260$ queries were evaluated for each algorithm on the same graph. Each query was measured in ten independent trials. Within each timing measurement, the C++ engine repeated the algorithm 10,000 times after graph loading; the reported execution time is the in-engine elapsed time divided by 10,000.

The following metrics should be recorded:

TABLE I
Final graph characteristics

Characteristic	Measured value
Vertices $\|V\|$	36
Edges $\|E\|$	39
Graph type	Weighted, undirected
Weight unit	Illustrative km
Graph SHA-256 prefix	`46a2f7ce089f837c`

- **Path cost:** sum of edge weights along the returned path.
- **Path validity:** whether every consecutive pair is an edge in G.
- **Explored vertices:** number of vertices settled or expanded, using one consistent definition for both algorithms.
- **Execution time:** elapsed algorithm time measured inside the engine, excluding browser rendering and, unless explicitly stated, process startup and API overhead.

The benchmark script launched the engine for each query and algorithm, validated the returned path against the graph, and compared the resulting path cost with Dijkstra. The engine parses the graph before starting the repeated algorithm timer; process startup, graph parsing, API handling, and browser rendering are excluded from the reported execution time. Mean and sample standard deviation were calculated over the ten trial measurements per query and aggregated across the fixed query set. Because the graph is small and the measured times are in the microsecond range, these timings are implementation- and machine-dependent; settled-vertex counts and correctness results are reported alongside time.

C. Experimental Results

The final graph characteristics and benchmark results are summarized in Tables I and II.

VIII. Discussion

Both algorithms completed all 1,260 queries and returned paths that passed edge-by-edge validation. No reachability mismatches or shortest-path cost mismatches were observed. These results provide experimental evidence that the two implementations agree on this fixed graph and query set; they are not a proof of correctness for arbitrary graphs.

Bidirectional Dijkstra settled an average of 13.594 vertices, compared with 19.000 for Dijkstra, a reduction of approximately 28.45%. However, its mean measured execution time was 0.001566 ms, compared with 0.001204 ms for Dijkstra. Thus, Bidirectional Dijkstra was approximately 30.0% slower relative to Dijkstra's mean time in this experiment, despite settling fewer vertices. The additional bookkeeping and coordination between two search frontiers can outweigh the reduction in settled vertices on a small graph. These timings are specific to this implementation, graph, compiler build, and machine; they do not establish a general performance ranking.

Visual route is something entirely different from shortest path correctness. While it could be the optimal route on the

TABLE II
ALGORITHM COMPARISON ON THE FIXED QUERY SET

Metric	Dijkstra	Bidirectional Dijkstra	Comparison / notes	Evidence
Queries completed	1,260	1,260	All ordered distinct pairs	Benchmark
Path validity (%)	100.00	100.00	0 invalid paths	Validator
Path-cost mismatches	0	0	Compared with Dijkstra	Validator
Mean explored vertices	19.000	13.594	Settled vertices; bidirectional sums both searches	Benchmark
Mean execution time (ms)	0.001204	0.001566	Mean over trials; 10,000 repetitions per measurement	Benchmark
Execution-time SD (ms)	0.000379	0.000415	Standard deviation	Benchmark
Unreachable queries	0	0	Reachability agreed	Validator
Trials per query	10	10	Same query set	Benchmark

graph, it could be wrong on the campus if an edge is missing, there is an extra edge or a node is positioned at the wrong map coordinates.

IX. LIMITATIONS AND THREATS TO VALIDITY

The prototype employs a schematic network for pedestrians. The locations of nodes, their connectivity and weights must independently be verified. Sampled weights do not justify claims regarding actual distances traveled and times taken. The present analysis is restricted to a given graph and two algorithms only. The findings from this analysis cannot be generalized to any campus network or large road networks.

Time analyses performed on small graphs are sensitive to timer resolution, compiler optimization, and system load. Repeating the algorithm 10,000 times per measurement reduces timer-resolution effects, but does not remove all measurement noise. The explored-vertex metric for Bidirectional Dijkstra is the sum of vertices settled by both searches. The evaluation covers one fixed graph and one implementation environment; the results should not be generalized to other campuses or large transportation networks.

X. CONCLUSION AND FUTURE WORK

In this research paper, AstraNav is presented as an educational prototype of a campus navigation system that incorporates a weighted graph, two shortest-path algorithms, C++17 engine, FastAPI backend, and browser-based visualization of the computed route. This system offers a real-world case to study greedy search for shortest paths, bidirectional search, path reconstruction, complexity, and evaluation of experiments.

In the all-pairs experiment on the 36-vertex schematic graph, both algorithms returned valid paths and agreed on shortest-path costs for all 1,260 queries. Bidirectional Dijkstra settled fewer vertices on average, but Dijkstra achieved the lower mean execution time in this implementation. The experiment therefore illustrates that a reduction in search effort does not necessarily translate into lower wall-clock execution time on a small graph. Future work includes surveying pedestrian distances, validating every walkway connection and map coordinate, expanding the graph, and evaluating larger synthetic networks and additional routing algorithms.

ACKNOWLEDGMENT

Acknowledgments, if applicable, should be added only for contributions or support actually received.

REFERENCES

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improving network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.

[3] A. V. Goldberg and C. Harrelson, "Computing the shortest path: A* search meets graph theory," in *Proc. 16th Annual ACM-SIAM Symposium on Discrete Algorithms*, 2005, pp. 156–165.

[4] H. Bast *et al.*, "Route planning in transportation networks," in *Algorithm Engineering: Selected Results and Surveys*, 2016, pp. 19–80.